Donato Marchionda. Manuel Cilli

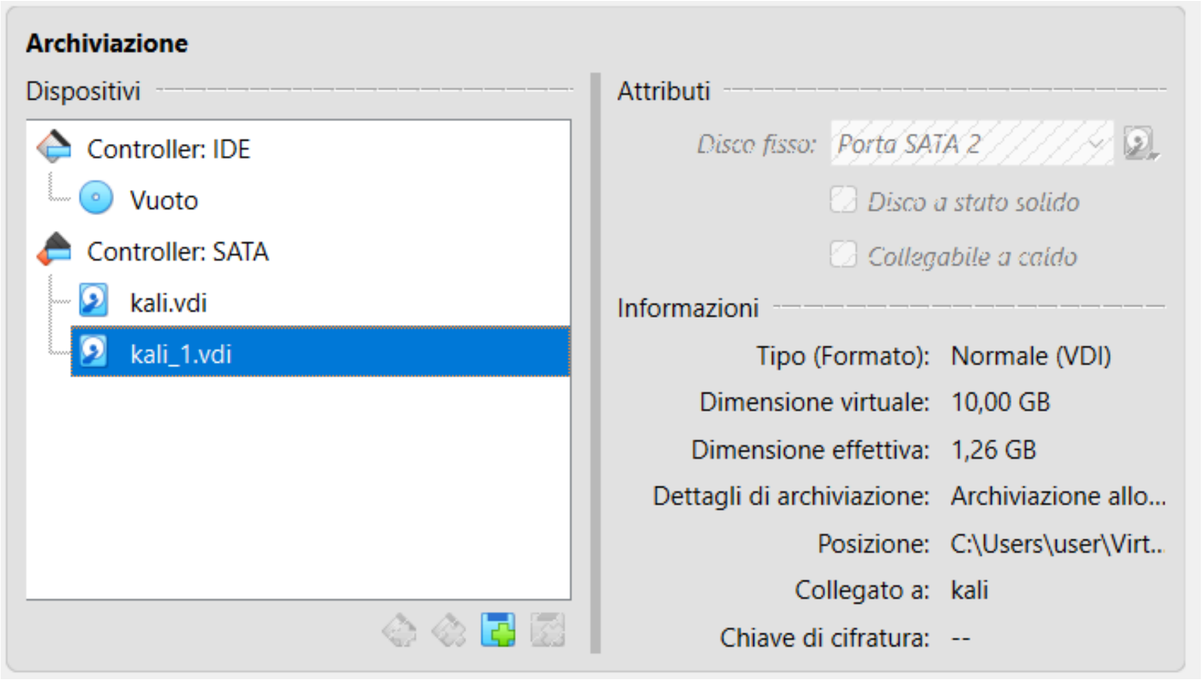
CRITTOGRAFIA
STORAGE AT REST

PRESENTATION TEMPLATE

❯ Start Slide

Realizzazione di una soluzione storage encryption at rest e benchmark tabellare.

Questa esercitazione si focalizza sull'implementazione della crittografia "at rest" (dati a riposo) e sulla valutazione del suo impatto prestazionale tramite benchmark. L'obiettivo è proteggere i dati memorizzati fisicamente sul disco da accessi non autorizzati in caso di furto dell'hardware o accesso al file system.



Architettura del Sistema

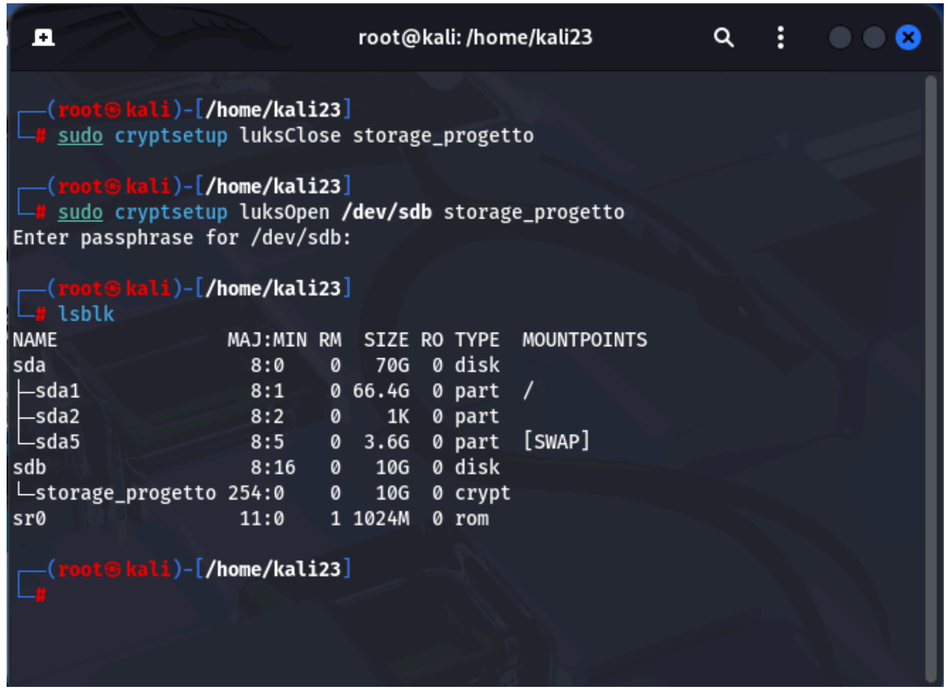
Il laboratorio si è basato su una struttura a tre strati:

1. **Hardware Layer: Un disco virtuale secondario da 10GB (/dev/sdb).**

2. **Encryption Layer: Utilizzo di LUKS/dm-crypt per la cifratura dei blocchi.**

3. **Filesystem Layer: Formattazione EXT4 per la gestione dei file leggibili.**

Operazioni (Step-by-Step)



A. Preparazione e Cifratura

- **Provisioning:** Aggiunto un disco VDI in VirtualBox.
- **Inizializzazione:** Eseguito **cryptsetup luksFormat /dev/sdb.**
  Questo ha creato l'header LUKS che protegge la Master Key tramite la tua passphrase.
- **Apertura:** Creato il device mapper **storage_progetto** che decifra i dati in tempo reale nella RAM.

About The Plan

Operazioni (Step-by-Step)

B. Configurazione Storage

- **Formattazione:** Creato il file system con **mkfs.ext4.**
- **Mounting:** Collegato il volume a **/mnt/crypt_data.**

About The Plan

```
┌──(root㉿kali)-[/home/kali23]
└─# sudo mkfs.ext4 /dev/mapper/storage_progetto
mke2fs 1.47.2 (1-Jan-2025)
Creating filesystem with 2617344 4k blocks and 655360 inodes
Filesystem UUID: 75e62563-b295-4850-9e79-138978a0eb2d
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

Formattazione:

Il Passaggio: Creazione del File System

(Comando eseguito: sudo mkfs.ext4 /dev/mapper/storage_progetto)

Spiegazione:

- **mkfs.ext4**: Sta per "Make File System" di tipo EXT4. È lo strumento che scrive le tabelle degli indici sul disco.

- /**dev/mapper/storage_progetto**: Questo è il punto fondamentale. **NON** abbiamo formattato /dev/sdb direttamente. Abbiamo formattato il "tunnel" decifrato.
  - Se avessi formattato direttamente /dev/sdb, avresti cancellato la cifratura LUKS.
  - Formattando il mapper, i dati del file system passano attraverso il motore di cifratura e arrivano sul disco fisico già protetti.

About The Plan

Mounting:
Cosa succede subito dopo?

(Comando eseguito: sudo mount /dev/mapper/storage_progetto /mnt/crypt_data)

Spiegazione:

Dopo questo comando, il disco è pronto ma "volante" nel sistema. Per usarlo davvero come una cartella, se deve dare il comando di Mount. Senza la formattazione (mkfs.ext4), il comando mount avrebbe dato errore dicendo: "Tipo di file system sconosciuto o mancante".

C. Benchmark Prestazionale

Abbiamo testato la velocità di scrittura con il comando **dd** (blocco da 1GB, scrittura sincrona).

```
┌──(root㉿kali)-[/home/kali23]
└─# sudo mkdir /mnt/crypt_data

┌──(root㉿kali)-[/home/kali23]
└─# sudo mount /dev/mapper/storage_progetto /mnt/crypt_data

┌──(root㉿kali)-[/home/kali23]
└─# sudo dd if=/dev/zero of=/mnt/crypt_data/testfile bs=1M count=1024 oflag=dsyn
c
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 5.61137 s, 191 MB/s

┌──(root㉿kali)-[/home/kali23]
└─# sudo dd if=/dev/zero of=~/testfile_plain bs=1M count=1024 oflag=dsync
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 3.19474 s, 336 MB/s

┌──(root㉿kali)-[/home/kali23]
└─#
```

Storage	Velocità Riscontrata	Impatto (%)
Standard (No Crypto)	336 MB/s	0% (Riferimento)
Cifrato (LUKS)	191 MB/s	-43.1%

Analisi: Il calo è dovuto all'overhead della CPU, che deve processare ogni bit tramite l'algoritmo AES-256 prima di scriverlo fisicamente.

About The Plan

D. Messa in Sicurezza:

Step 1: Uscita dalla directory occupata
Prima di tutto, dato che eri "dentro" la cartella /mnt/crypt_data, hai dovuto spostarti fuori per non bloccare il sistema:
Bash: comando **cd ~**

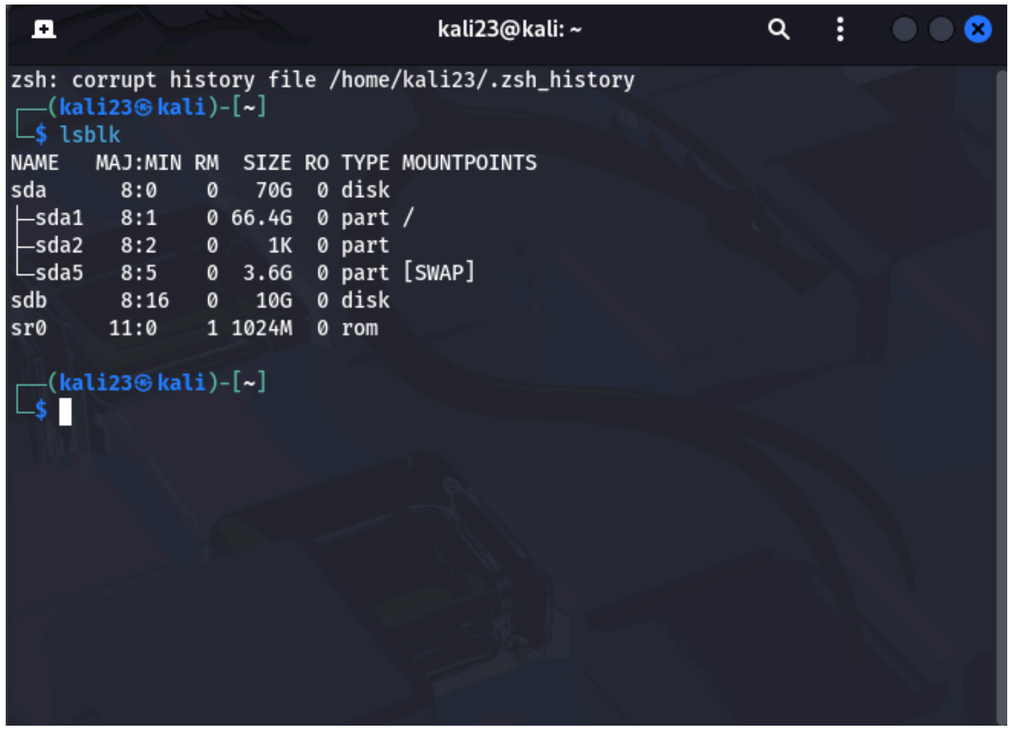
Step 2: Smontaggio (Unmount)
Dare il comando per scollegare il file system dalla cartella di Kali. Per risolvere il blocco del kernel, abbiamo usato la versione "Lazy" (pigra):
Bash: comando **sudo umount -l /mnt/crypt_data**

- Cosa ha fatto: Ha rimosso il collegamento logico tra la cartella e il disco cifrato. Da questo momento, i file non sono più visibili nel sistema.

Step 3: Chiusura del Mapping LUKS
Questo è il comando che "sigilla" la cassaforte. Senza questo, il disco sarebbe rimasto sbloccato in memoria:
Bash: comando **sudo cryptsetup luksClose storage_progetto**

Step 4: Verifica finale
Hai controllato che tutto fosse andato a buon fine con:
Bash: comando **lsblk**

- Risultato: Hai visto che sotto sdb non c'era più nulla. La struttura a "ramo" era sparita, confermando che il volume è ufficialmente At Rest.



About The Plan

Conclusioni Progetto

Il progetto è terminato con successo, si è dimostrato:

Punto 01

Configurazione un'architettura dm-crypt/ LUKS

Punto 02

Gestione dei File System su volumi logici cifrati.

Punto 03

Condurre test prestazionali comparativi e analizzarne i risultati.

Conclusione:

Cybersecurity e Data-at-Rest
Il progetto ha dimostrato che la crittografia LUKS è una difesa impenetrabile per la riservatezza dei dati.

Nonostante un overhead prestazionale del 43%, il sacrificio in termini di velocità è ampiamente giustificato. In un contesto di cybersecurity moderna (e conformità GDPR), la protezione del dato a riposo è l'unica garanzia contro il furto fisico dei supporti, trasformando un potenziale data breach in un ammasso di bit inutilizzabili per l'attaccante.